CITS5501 Project Phase 1 – Group 16

Tyson Haines (23779585), Akshay Raghuwanshi (24991278), Khushi Sesetti (23823678), Kevin Tan (23789621), Celine Xu (24714078)

August 2026

1. Introduction

This is the document for the CITS5501/CITS3501 project phase 1. It details our Collaboration & Workflow Plan (Task 1) and our answers to the Phase 1 report questions (Task 3), covering how we will organise ourselves, our choice of implementation language and tooling, the technical risks we anticipate in building a web scraper for an online policy library, and how we plan to approach testing and code quality ahead of Phase 2.

Our group repository is hosted at: github.com/TysonHaines/CITS5501-Group-16.

2. Team members

Name	Student number	GitHub username
Tyson Haines	23779585	TysonHaines
Akshay Raghuwanshi	24991278	AkshayRaghuwanshi
Khushi Sesetti	23823678	khushisesetti
Kevin Tan	23789621	phattydumpling
Celine Xu	24714078	1031Celinela

3. Collaboration & Workflow Plan

3.1. Team Communication & Responsibilities

We will communicate primarily via Microsoft Teams, using a dedicated channel for day-to-day messages and file sharing, and Teams calls for any online meetings.

We will meet weekly, or more frequently if needed, especially when approaching a deadline. These meetings will be either in person or online via Teams.

Accountability and progress tracking will be handled through GitHub Issues: tasks are logged as issues, assigned to an individual (for small tasks) or multiple group members (for complex tasks), and reviewed at each weekly meeting. This will help the group explore multiple ideas and perspectives on a problem, and reduce risk of any singular member

becoming ill. Any task not progressing will be raised and reassigned or broken down at the next meeting.

At this stage, we will not assign permanent technical roles, as Phase 2 has not yet been released. Initial responsibilities have been recorded amongst the team as the following:

- Kevin: Group leader
- Tyson: Project coordinator/organiser
- Celine: Editor/Reviewer
- Akshay: Editor/Reviewer
- Khushi: Meetings notetaker/time keeper

Once Phase 2 begins, technical responsibilities will be distributed across the team, in a way which would best suit the team based on our strengths and weaknesses, as well as familiarity with programming languages.

3.2. Version Control Strategy

Our project is hosted on GitHub, in a private repository at: github.com/TysonHaines/CITS5501-Group-16.

- We will use a feature-branch workflow: `main` is kept in a working state, and all changes are made on feature branches (e.g. `feature/`) opened against an issue.
- Merges into `main` happen via Pull Request, and require review/approval from at least one group member other than the author before merging.
- Commit messages should be kept clear, and be detailed enough such that any member can understand the changes made. It should also reference the relevant issue number where applicable (e.g. `Fixes #12`).
- CI checks (tests, linting, type-checking) must pass before a PR can be merged.

Our group name will be: 404 Brain Not Found

3.3. Resource Risk Management

The main resourcing risk we anticipate is a group member becoming unavailable (e.g. illness, other assessment load) for a period during the project. To mitigate this:

- Tasks are tracked as GitHub Issues rather than held only in one person's head, so anyone can pick up an unfinished task if needed.
- We avoid having only one person understand any single component - pull requests are reviewed by another member, which spreads familiarity with the codebase across the group. This is reinforced by having members collaborate across features where practical, so knowledge of important features of the project is not restricted to any single person.
- If a member is unavailable for an extended period, and given notice is provided, the group will redistribute their open issues at the next weekly meeting and, if necessary, flag the situation to our facilitator.

- We will aim to finish core functionality with some buffer before the deadline, rather than leaving all work to the final days, to absorb the impact of any single person being unavailable.

4. Report

Planning and organization

1. **Where is your group's Collaboration & Workflow Plan maintained? Is it in the Git repository, or elsewhere?**

The Collaboration & Workflow Plan will be maintained in our GitHub repository at github.com/TysonHaines/CITS5501-Group-16, so it is version-controlled alongside the rest of our project, in case of any changes to the document.

Additionally, there will be a copy of this pinned within the Teams chat,

2. **What tool or approach are you using to track assigned tasks and completion? Why did you choose it? Has anyone in the group used it before?**

We are using GitHub Issues, together with a GitHub Projects (Kanban-style) board. We chose it because it lives alongside our codebase and pull requests, so there is no need to keep a separate spreadsheet or board in sync, and issues can be directly referenced from commits and PRs.

We have chose this because everyone in the group also has at least some previous experience with GitHub, which will make the initial processes transition smoothly with getting everyone on board. Whilst some members have not used GitHub Projects, this will be taught through self learning and other members of the group, and the learning curve is expected to be manageable.

Language and tooling

3. **Which language has your group chosen? Have any members of the group not used it – and if so, how will they get familiar with it?**

We have chosen Python as the language that the group will use for this project. From a group perspective, everyone in the group is familiar with Python and at least some relevant tools and libraries that Python provides. Python also has widely used libraries for HTTP requests, HTML parsing and document processing, which makes it well suited to the requirements of this project.

Ranking of factors influencing our choice (most to least important):

1. Familiarity with the language - most/all of the group have used Python before, which reduces time spent learning relevant skills for this project with a different language.
2. Library support - From initial research, Python provides mature libraries for web scraping, including HTTP requests, HTML parsing, and PDF/Word/RTF document

handling (e.g. `requests`, `pypdf`, `python-docx`, `striprtf`), which relates to the scope of this project. Additionally, Python allows extraction of unstructured data, and manipulate it using `pandas`, which can be used in machine learning models and databases (however, this is likely beyond the scope of this project).

3. Ease of debugging - On top of the group already familiar with Python, Python is easier to debug compared with Java or C, as it is an interpreted, dynamically-typed language with a highly flexible ecosystem built specifically for handling erratic web data. Web scraping is inherently messy because external website structures change constantly, content breaks, and data formats are unpredictable. Python is also interactive in nature (e.g. REPL), making it easy to probe a malformed page or document interactively while developing the scraper.

- **Advantages:** Python's library ecosystem for web scraping and document parsing is a major advantage for this project, and its dynamic typing supports fast iteration while experimenting with an unfamiliar website.
- **Disadvantages:** The main disadvantage is that dynamic typing can hide bugs that a compiler would otherwise catch, which is why we plan to use a type checker (see Section 4.5).

If we had to implement the project in Java, document parsing and quick experimentation with different web responses may be more cumbersome, as Java is generally more verbose and has a stricter type system. However, Java's static typing could catch some errors earlier and may make parts of a larger codebase easier to reason about and maintain.

4. **What build/dependency management tool will you use (e.g. uv or poetry for Python; Gradle or Maven for Java; something else)? Has anyone in the group used it before – or will this be the first time setting up a project with it?**

We will use `uv` for dependency and environment management.

Our group decided to choose this tool because it is specified as an installed package in the test environment, providing a straightforward way to manage dependencies and is easily reproducible in the test environment. `uv` has also been previously used by some members of the group, whilst members who have no used it before will learn through practically testing the tool, reading documentation of the tool and asking questions to members to have used it before, to ensure that they will have sufficient knowledge to use for the project.

Technical risks

5. **What parts of a web scraper do you expect could most easily fail, and how?**

The most fragile parts are likely to be:

- The HTTP layer (timeouts, connection resets, unexpected status codes such as 429/503 from rate limiting).

- HTML parsing (the site's markup structure changing, or pages varying slightly between documents so that a selector that works on one page fails on another)
- Document parsing (PDF, Word, and RTF files that are malformed, password protected, or use encodings our parser doesn't expect).
- Redirects and broken links are also an easy source of failure, since a scraper that assumes every link resolves to a valid document will break the moment one doesn't.
- Client-side rendering is another likely failure point: if the library (or parts of it, e.g. a document list, pagination, or a "load more" button) is rendered or populated by JavaScript after the initial page load, a scraper that only fetches and parses the raw HTML response will simply not see that content - it never runs the script that would have inserted it. The same applies to content that only appears after a user action, such as expanding an accordion, switching a tab, or scrolling to trigger infinite loading. Since the raw response still returns a valid 200 status with a well-formed (if incomplete) page, and since the scraper won't error, it will just silently return less data than expected.

6. **Which external factors could make your software stop working after it has been completed?**

A redesign of the site's HTML structure or URL scheme, changes to stricter rate, limiting/ IP blocking, the site moving to a login-gated or JavaScript-rendered version of the library, TLS certificate changes, or upstream dependency updates (e.g. a new major version of a parsing library) that change behaviour or drop support for the selected Python version.

7. **Which components of your project do you think will be easiest to test? Which do you expect will be hardest?**

As the full specs for phase 2 are not yet provided, we are only able to assume the following, based on the existing description provided in phase 1.

- Easiest: the parsing/extraction logic and the query logic, since these can be tested offline against a fixed set of sample HTML pages and sample documents, giving fully deterministic, repeatable tests.
- Hardest: anything that depends on live network behaviour, such as retry/backoff logic under rate limiting, or handling of genuinely unexpected server responses. These are hard to reproduce reliably without a large captured corpus of real (and possibly malformed) responses.

8. **What assumptions is your software likely to make about the website it scrapes? How could those assumptions be violated?**

Likely assumptions include:

- The site's HTML structure and CSS selectors remain stable over time.
- Documents are served with correct/consistent content-type headers, tags and character encoding.

- The site is reachable over HTTPS with a valid certificate; content is present in the initial HTML response rather than injected by JavaScript; and no authentication is required to reach policy documents.

Each of these could be potentially violated: a site redesign breaks selector assumptions, a mislabelled content-type could cause a PDF to be parsed as HTML, and a move to a JavaScript rendered front end would break a scraper that only fetches raw HTML. If any of these become assumptions that our implementation depends on during Phase 2, we will document them using the numbered assumption format required by the project brief.

Team practices

9. **Suppose a serious bug is discovered two days before the deadline. How would your team investigate and respond?**

First, we would write a failing test case(s) that reproduces the bug, so the fix can be verified objectively. We would triage its severity, i.e. does it affect core, marked functionality, or an edge case, and assign this to whoever is most familiar with the affected component. The fix would be made on a branch and go through our usual PR review before merging, even under time pressure, to avoid introducing further bugs as a result. If it can't be safely fixed in time, we would document it as a known limitation/assumption rather than merge an untested fix.

10. **What will convince your team that a feature is "finished"?**

A feature is considered finished when it satisfies the following requirements relevant to the project:

- Contains appropriate unit, and where relevant, integration tests covering both success and failure paths.
- Passes linting and type-checkers - Once it has been reviewed and merged via a pull request.
- Public methods have documentation blocks per the project brief.
- Manually checked against the relevant requirement in the brief.

11. **If two group members disagree about whether software is "working correctly", how will you resolve that disagreement?**

To resolve disagreement, we will first check against the project specification and gain clarification from the teaching staff to determine the intended behaviour.

If the expected behaviour is clear, we will write a concrete test case that captures it and use that test to assess the implementation. If the specification is genuinely ambiguous, we will discuss the different interpretations as a group and, if needed, ask the teaching staff for clarification rather than choosing one interpretation arbitrarily. Appropriate assumptions will be made accordingly alongside this.

Once the expected behaviour is agreed, we will create relevant tests, so future changes can be checked consistently.

Code quality

12. **Will you use a type checker, and if so, will it run in CI or just periodically/ manually? Has anyone used one before? Do you plan to use a linter?**

Our team will use a type checker. We plan to use `mypy` for static type checking and `pylint` for linting. This is a fairly common type checker and linter for Python, meaning plenty of support will be available. We will have this automated, through GitHub Actions and it will be run in CI on every pull request as a required check, rather than manually to reduce the risk of human error. Most members of the group are unfamiliar with type checkers, so we will get up to speed by running it locally against the early Phase 2 codebase and asking questions where required.

13. **Suppose a group member's contribution doesn't pass your linting or type checking checks. What will be the consequences? Will merges be blocked if these checks fail, or will the checks be advisory only?**

Once CI is set up, passing lint and type-checking will be a required check on pull requests - a PR cannot be merged into `main` while these checks are failing. This will be automatic, once GitHub Actions is set up properly. Before CI is set up, checks are advisory and expected to be run locally before opening a PR.

Testing framework

14. **What test framework will you use (e.g. pytest for Python; JUnit 5 for Java; something else)? Has the group used it before, or only heard of it?**

We will use pytest as the testing framework for the project. This is reasonable given that we have chosen to complete the project using Python. Within the group, some members have previously used pytest, whilst others have only heard of it.

CI / automation

15. **Will you set up continuous integration (e.g. GitHub Actions) to run your test suite automatically, or run tests manually before submission? Have any members of your group used continuous integration before?**

We plan to set up GitHub Actions to run our pytest suite, linting and type-checking on every pull request. Few members of the group have had small amounts of prior experience with CI, while completely new to most others. Everyone in the group will gain further understanding of CI through self and collaborative learning and by helping configure and review the workflow during Phase 2.

Testing mindset

16. **Suppose your scraper is finished and submitted. Now imagine another student is trying to break your software (get it to exhibit failures or unexpected behaviour). What might they try first?**

If another student is attempting to break the software, they would begin with some common entry entry points (perhaps challenges from when they created their own webscraper). This could include feeding it malformed or unexpected URLs, pointing it at documents with unusual encodings or corrupted PDF/Word/RTF files, triggering rate limiting by making rapid requests, simulating a slow or non-responding server (timeouts); submitting empty search queries, and following links that redirect in a loop or point to a 404.

17. **How will you know that your software is producing the correct output, rather than merely some output?**

We will test against known expected outputs. This means that for a fixed set of sample pages/documents, we know in advance what the correctly extracted text/metadata should be, and assert our output matches it exactly (or matches expected structure), rather than just checking that the program didn't crash. For query functionality, we will assert that known documents are returned for known queries, not just that *some* result is returned.

18. **What evidence would increase your confidence that your software is reliable?**

High test coverage across both success and failure/edge cases, consistently passing CI runs, integration tests run against recorded/fixture server responses (so failure-handling paths are exercised deterministically), code review from another group member, and some manual exploratory testing against the live site.

19. **At this stage of the project, what do you think will be the biggest challenge in producing reliable software?**

The biggest challenge is likely to be the unpredictability of a real external website. We can't fully anticipate every way pages, documents, or server behaviour might deviate from what is expected until we've actually explored the site, so our error-handling and test suite will need to evolve as we discover more failure modes during Phase 2.

Additionally, since it is the first time working with a webscraper for many members of the group, familiarity with the tools we use listed in previous questions will also be a challenge in ensuring that the software is completely reliable, so making sure that the correct libraries are used for its purpose will also play a role in producing reliable software.

Process and standards

20. **What will be your code review process - does someone other than the author review changes before merging, or is this informal?**

All changes go through a GitHub pull request and must be reviewed and approved by at least one group member other than the author before merging into `main`. Review is done asynchronously via PR comments where possible. For urgent fixes or if approaching a deadline, review may happen synchronously over a quick Teams call instead.